Stream ciphers

- A class of symmetric ciphers (same keys) 2 categories of stream ciphers
  - stream, block
- Encrypts plaintext *s* bits at a time using a random key stream

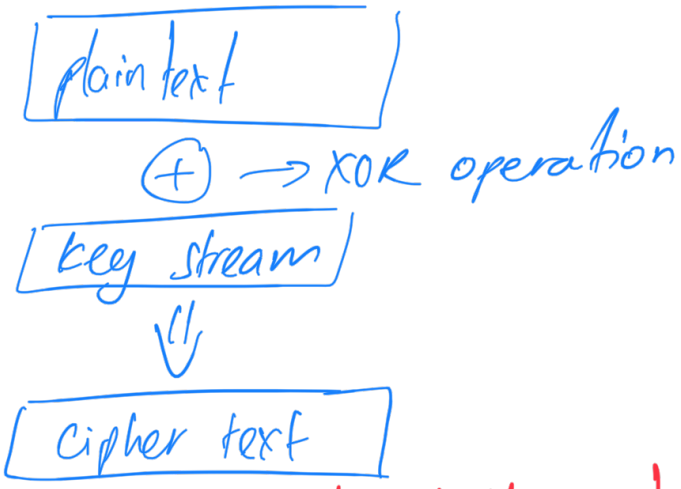
single byte or single bit

$s = 1$ or $s = 8$

$P_i \oplus k_i = C_i$

encrypting: $P \oplus k = C$

decrypting: $C + k = P$



- One-time pad (OTP) → The only unbreakable cipher.
  - key is as long as the plaintext
  - key is completely random (not using any algorithm)
  - key is only used once

- One-time pad (OTP)

  - The only unbreakable cipher
  - Every other encryption algorithm is theoretically breakable.

  - Why is it unbreakable?
  - Given any plaintext of length equal to the ciphertext, there is a key that will produce that plaintext

⇒ all keys are equally likely

  - Limitations

→ Not easy to generate large quantities of random bits
→ not easy to do the key distribution

∴ The goal of cryptographers is to simulate the OTP but with a key that has a practical key size.

OTP used for:
→ used for small plaintexts & plaintext that is very high-security

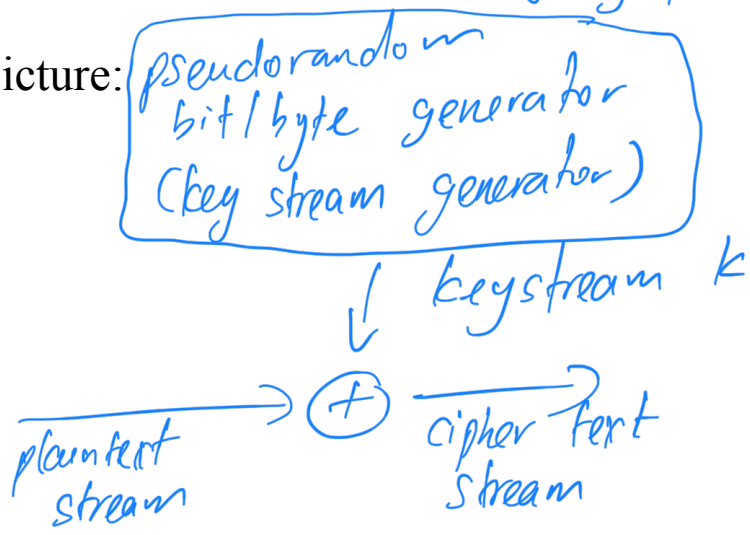
What can be done to deal with the OTP limitations?

- Encrypt plaintext *s* bits at a time using the output of a key stream generator that takes a key K as input

generator takes a key as input

key K

Picture:



Required properties:

$P_1 \oplus k = C_1$

$P_2 \oplus k = C_2$

$C_1 \oplus C_2 = P_1 \oplus P_2$

Assume that the attacker knows that P_1 & P_2 have been encrypted with the same k.

Assume that the attacker knows everything except the actual value of the key

- Key stream should have a large period
  - not repeat for a long time
- Key stream should look like a random sequence
  - to all statistical tests.
- The key K should be long enough
  - to preclude exhaustive search of k (128 bits or longer)

Some possibilities

- RC4
- Salsa20

RC4 keylength: 1–256 bytes
i.e. 8 – 2048 bits

← SSL, WEP, WPA

RC4: stream cipher designed by Rivest in 1987

S & T have the same length.

```
/*  Initialization */
for i=0 to 255 do
    S[i] = i;
    T[i] = K[i mod keylen];
```

S[0 1 1 ... 1255]

↳ whatever length

```
/* Initial Permutation of S */
j=0;
for i=0 to 255 do
    j = (j + S[i] + T[i]) mod 256;
    Swap (S[i], S[j]);
```

```
/* Stream Generation */
i, j = 0;
while(true)
    i = (i + 1) mod 256;
    j = (j + S[i]) mod 256;
    Swap (S[i], S[j]);
    t = (S[i] + S[j]) mod 256;
    k = S[t];
```

Generate keystring.

(see RC4 diagram) Figure 7.6 RC4

⇒ Stream Generation.

RC4 features:

- very simple
- period : $> 10^{100}$
- very fast

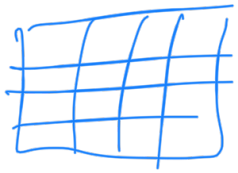
RC4 limitations:

- biases in the first few bytes of keystring (some bytes were more likely to occur than other bytes). ⇒ people could discard, say the first 1000 bytes
- recent attacks
  ⇒ deprecated, prohibited from use in all versions of TLS

Salsa20

- One of the finalists in the eSTREAM competition in Europe (2004-2008)
  - s/w preference
    ↳ software

- How does it work?
  - uses 32-bit words
  - state: 8 words of key (8 x 32 = 256 bits)
    2 words of nonce (random - value that wasn't used for any other plaintext).
    2 words of position (where in the plaintext stream you're going to be using this key)
    4 fixed words (4 constant values that will always be used).



16 words (512 bits), arranged as a 4x4 matrix

- (see diagram of quarter-round function)

20 rounds
  - odd numbered rounds (1, 3, ..., 17, 19): quarter-round function on columns
  - even numbered rounds (2, 4, ..., 20): quarter-round function on rows
  - At the end of 20 rounds, this matrix is completely permuted.
  - After 20 rounds, take resulting matrix and XOR with original matrix. The result is the key stream (512 bits) that is XORed with 512 bits of plaintext.

- Efficient and still secure
  - Block ciphers are more popular than stream ciphers